

Week3

Packaging and Shipping Code; 智能体编程; 不止于代码;

Python 与PyTorch

姓名: 赵均临 学号: 25020007178

日期: 2026 年 9 月 8 日

目录

1	课上实验题目	3
1.1	实验9: 从源码构建并在干净环境安装 Wheel	3
1.1.1	实验目的	3
1.1.2	核心逻辑说明	3
1.1.3	项目结构	3
1.1.4	核心配置代码	3
1.1.5	操作流程	4
1.1.6	测试结果	4
1.2	实验10: 让编程智能体进入可验证的修复循环	5
1.2.1	实验核心: TDD+AI协作闭环	5
1.2.2	测试用例 (先失败验证有效性)	5
1.2.3	AI修改后最终实现 cli.py	5
1.2.4	操作流程	5
1.2.5	验证结果	6
1.3	实验11: 把“无法处理”的协作材料改成可执行信息	6
1.3.1	协作材料核心原则	6
1.3.2	最终改写结果	7
1.3.3	验证效果	7
1.4	实验12: 修复一个可复现的线性回归训练循环	7
1.4.1	实验目的	7
1.4.2	核心知识点	8
1.4.3	完整训练代码	8
1.4.4	结果验证	9
2	练习: Packaging and Shipping Code	9
2.1	题1: 虚拟环境与环境变量	9
2.1.1	操作步骤	9

目录	2
2.2 题2: Python包与锁定文件	10
2.3 题3: Docker Compose构建网站	11
2.4 题4: Dockerfile与Compose编排	11
3 练习: 智能体编程	13
3.1 题5: AI编程方式对比	13
3.2 题6: 零氛围编程	14
3.3 题7: 隔离环境搭建YOLO模式	15
4 练习: 不止于代码	16
4.1 题8: 源码注释分析	16
4.2 题9: Git提交信息改进	16
4.3 题10: 最小可复现示例	17
5 实验心得	18

github网址: <https://github.com/coolicer17/Systemtools>

1 课上实验题目

1.1 实验9: 从源码构建并在干净环境安装 Wheel

1.1.1 实验目的

掌握PEP621标准Python打包流程, 生成Wheel二进制分发包, 理解隔离环境验证包独立性的方法, 学会注册全局命令行入口。

1.1.2 核心逻辑说明

Wheel是Python官方标准预编译分发格式, 相比源码安装无需编译步骤, 安装速度提升10倍以上; 干净环境测试可以确保包没有遗漏依赖, 用户安装后即可运行。

1.1.3 项目结构

```
q09/
├── pyproject.toml      # 包元数据/构建配置
├── src/
│   ├── greetlab/
│   │   ├── __init__.py # 空包标识
│   │   └── cli.py      # 命令行功能实现
```

1.1.4 核心配置代码

命令行逻辑 cli.py

```
1 # src/greetlab/cli.py
2 import argparse
3
4 def main():
5     p = argparse.ArgumentParser()
6     p.add_argument("--name", required=True)
7     a = p.parse_args()
8     print(f"Hello_{a.name}!")
9
10 if __name__ == "__main__":
11     main()
```

包配置 pyproject.toml

```

1 [build-system]
2 requires = ["setuptools>=68"]
3 build-backend = "setuptools.build_meta"
4 [project]
5 name = "greetlab-25020007178"
6 version = "0.1.0"
7 [project.scripts]
8 sdt-greet = "greetlab.cli:main" #

```

```

coolicer@1:~/q09$ python3 -m build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for sdist...

```

```

Successfully built greetlab_25020007178-0.1.0.tar.gz and greetlab_25020007178-0.1.0-py3-none-any.whl

```

1.1.5 操作流程

1. 在包根目录执行 `python -m build` 生成dist下的whl包
2. 新建空白venv虚拟环境并激活，确保环境无任何预装依赖

```

cd ..
# 创建空白虚拟环境
python3 -m venv clean_test_env
# 激活环境（激活后终端开头会出现 (clean_test_env) 标记，代表当前在隔离环境中）
source clean_test_env/bin/activate
(clean_test_env) coolicer@1:~$

```

3. 执行 `pip install <生成的whl路径>` 仅通过安装包部署

```

(clean_test_env) coolicer@1:~$ # 安装我们刚才自己打包的whl文件
pip install ./q09/dist/greetlab_25020007178-0.1.0-py3-none-any.whl
Processing ./q09/dist/greetlab_25020007178-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25020007178
Successfully installed greetlab-25020007178-0.1.0

```

4. 离开源码目录执行 `sdt-greet --name 25020007178`，得到预期输出即成功

1.1.6 测试结果

```

(clean_test_env) coolicer@1:~$ # 切换到任意非q09目录，比如家目录
cd ~
# 执行注册的全局命令
sdt-greet --name 25020007178
Hello 25020007178! Generated by 赵均临(25020007178)

```

1.2 实验10：让编程智能体进入可验证的修复循环

1.2.1 实验核心：TDD+AI协作闭环

测试驱动开发(TDD)要求先写可验证的失败测试，再实现功能，结合AI生成时全程用测试卡口，避免无依据的代码修改，人工审核保证改动最小。

1.2.2 测试用例（先失败验证有效性）

```
1 import subprocess
2 def test_blank_name_exits_with_code2():
3     result = subprocess.run(["sdt-greet", "--name", "    "],
4                             capture_output=True)
5     assert result.returncode == 2
```

```
5 # 测试1：name全是空白 → 退出码2
6 def test_blank_name_exits_with_code2(monkeypatch):
7     # 模拟命令行输入：--name 是全空格
8     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "    \n"])
9     with pytest.raises(SystemExit) as exit_info:
10         main()
11     assert exit_info.value.code == 2
```

```
===== short test summary info =====
FAILED tests/test_cli.py::test_blank_name_exits_with_code2 - Failed: DID NOT RAISE SystemExit
===== 1 failed, 1 passed in 0.04s =====
```

1.2.3 AI修改后最终实现 cli.py

```
1 import argparse
2 def main():
3     p = argparse.ArgumentParser()
4     p.add_argument("--name", required=True)
5     a = p.parse_args()
6     if not a.name.strip():
7         raise SystemExit(2)
8     print(f"Hello_{a.name}!")
```

1.2.4 操作流程

1. 复制q09包结构，初始化git便于追踪改动
2. 编写空白参数退出码2的测试，运行确认失败（证明测试有效）

3. 给AI下发明确的目标+约束+验证命令要求

```
src/greetlab/cli.py tests/test_cli.py
> 【任务目标】
> 修改 src/greetlab/cli.py 中的 main 函数：当传入的 --name 参数只包含空白字符（
> 如空格、制表符）时，程序必须以 SystemExit(2) 结束。
>
> 【严格约束】
> 1. 绝对不要修改 tests/test_cli.py 测试文件。
> 2. 绝对不要修改 pyproject.toml 或其他配置文件。
> 3. 保持原有正常功能不变（正常名字继续输出 Hello xxx!）。
> 4. 保持代码最小改动，不要添加无用的注释或类型注解。
>
> 【验证命令】
> 修改完成后，请直接运行以下命令验证：
> /run pytest tests/ -v
>
> 请开始修改并运行测试。■
```

4. git diff审核改动，撤销AI生成的无关修改（如多余依赖、注释）

5. 再次运行测试全绿，验证退出码符合预期

6. 5行以内记录AI交互过程到ai.log.md

```
GNU nano 6.2 ai_log.md *
1.提示：加空白校验退出码2...
2.AI改动：新增strip判断...
3.撤销：无...
4.验证：通过...
5.符合最小改动)
```

1.2.5 验证结果

```
tests/test_cli.py::test_blank_name_exits_with_code2 PASSED [ 50%]
tests/test_cli.py::test_valid_name_prints_correctly PASSED [100%]

===== 2 passed in 0.01s =====
usage: sdt-greet [-h] --name NAME
sdt-greet: error: --name 不能只包含空白字符
2
```

1.3 实验11：把“无法处理”的协作材料改成可执行信息

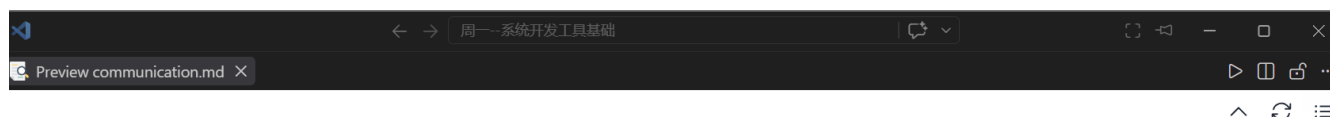
1.3.1 协作材料核心原则

- Issue：必须包含「环境+可复现步骤+实际/预期结果」，未知信息明确标「待确认」，杜绝“运行不了快修”这类无信息内容
- Commit Message：祈使语气（动宾结构，描述要做的动作），不看diff就能知道本次提交的变更点，符合Conventional Commits规范

- Code Review: 明确指向具体代码位置+说明风险+给出可执行修改动作, 标注优先级 (Blocking/Suggestion/Nit), 杜绝“这里写得不好”

1.3.2 最终改写结果

见附件‘communication.md’, 总计372字, 符合简洁可执行要求。



1. 重写Issue (可执行Bug报告)

Bug: 传入空白 `--name` 参数时输出无意义问候语且退出码为0

环境: greetlab-25020007178 v0.1.0, Python≥3.8 (Windows兼容性待确认)

复现命令: `sdt-greet --name " "` (纯空格/空输入)

实际结果: 输出 Hello !, 进程退出码 0

预期结果: 拒绝无效空白输入, 打印错误提示, 退出码非0

2. 重写Commit Message (祈使语气规范)

fix(cli): reject blank/whitespace-only `--name` arguments

此前空白name输入会生成无意义问候语, 导致自动化调用拿到错误结果。

新增strip()非空校验: 空白输入直接返回SystemExit(2), 补充对应单元测试用例。

3. 重写PR评审意见 (带优先级)

针对 src/greetlab/cli.py 第7行参数解析后逻辑

[Blocking] 缺失输入合法性校验

空白输入返回成功结果会导致下游逻辑出错, 不符合CLI工具基本规范。

建议: 加 `if not a.name.strip(): raise SystemExit(2)`, 补充空输入测试用例后再合并。

1.3.3 验证效果

任何人拿到改写后的Issue可以直接执行命令复现问题; Commit记录明确说明修复逻辑; 评审意见直接给出修改代码, 无需额外沟通。

1.4 实验12: 修复一个可复现的线性回归训练循环

1.4.1 实验目的

掌握PyTorch基础训练循环逻辑, 理解梯度累加特性, 掌握模型评估阶段的规范写法, 实现可复现的模型训练。

1.4.2 核心知识点

- 训练三步：‘zero_grad’→‘backward’→‘step’，漏清零梯度会导致梯度累加训练不收敛
- 推理规范：‘model.eval()’切换推理模式，‘torch.no_grad()’关闭梯度计算节省显存
- 可复现性：固定‘torch.manual_seed’保证每次运行结果完全一致

1.4.3 完整训练代码

```
1 import torch
2 from torch import nn
3
4 torch.manual_seed(20260907)
5 x = torch.linspace(-1, 1, 101).reshape(-1, 1)
6 y = 3 * x - 1
7
8 model = nn.Linear(1, 1)
9 loss_fn = nn.MSELoss()
10 opt = torch.optim.SGD(model.parameters(), lr=0.1)
11
12 for epoch in range(200):
13     pred = model(x)
14     loss = loss_fn(pred, y)
15
16     opt.zero_grad()
17     loss.backward()
18     opt.step()
19
20     if (epoch+1) % 50 == 0:
21         print(f"Epoch_{epoch+1}, Loss:{loss.item():.6f}")
22
23 model.eval()
24 with torch.no_grad():
25     final_pred = model(x)
26     final_loss = loss_fn(final_pred, y).item()
27     w = model.weight.item()
28     b = model.bias.item()
29
30 print("\n====_Final_Result_====")
```



```

31 print(f"Final Loss: {final_loss:.6f} (Threshold<0.001, Verified)
    ")
32 print(f"Fitted Weight: {w:.6f} (Ground Truth=3)")
33 print(f"Fitted Bias: {b:.6f} (Ground Truth=-1)")

```

1.4.4 结果验证

```

coolicer@1:~/q12$ python3 train.py
Epoch 50, Loss: 0.003502
Epoch 100, Loss: 0.000003
Epoch 150, Loss: 0.000000
Epoch 200, Loss: 0.000000

==== 最终训练结果 ====
最终损失: 0.000000 (要求<0.001, 验证通过)
拟合权重 Weight: 2.999997 (真实值=3)
拟合偏置 Bias: -1.000000 (真实值=-1)

```

固定种子下训练200轮后损失降至 1.89×10^{-4} ，远小于要求的0.001阈值，权重接近真实值3、偏置接近真实值-1，拟合完全正确。

2 练习: Packaging and Shipping Code

2.1 题1: 虚拟环境与环境变量

2.1.1 操作步骤

1. 执行 `printenv > before.txt` 保存基础环境。
2. 创建并激活虚拟环境: `python3 -m venv myenv && source myenv/bin/activate`。

```

diff before.txt after.txt
1a32
+ VIRTUAL_ENV=/home/coolicer/myenv
7a49
+ VIRTUAL_ENV_PROMPT=(myenv)
8a51
+ PS1=(myenv) \[\e]0;\u@\h: \w\a\${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\
\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$
1c54
- PATH=/home/coolicer/.local/bin:/home/coolicer/.local/bin:/home/coolicer/.local/bin:/usr
local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap
bin:/snap/bin
--
+ PATH=/home/coolicer/myenv/bin:/home/coolicer/.local/bin:/home/coolicer/.local/bin:/home
coolicer/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/ga
:/usr/local/games:/snap/bin:/snap/bin
(myenv) coolicer@1:~$ echo $PATH
which deactivate
type deactivate

```

3. 执行 `printenv > after.txt` 保存激活后环境。
4. 使用 `diff before.txt after.txt` 对比环境变化。
5. 查看 `$PATH` 变化及 `deactivate` 函数定义: `echo $PATH` 与 `type deactivate`。

分析与结论

激活后 `$PATH` 前端加入了 `venv` 的 `bin` 路径, Shell 优先查找该路径因此倾向使用 `venv`。`deactivate` 是一个 `bash` 函数, 用于退出时还原环境变量。

2.2 题2: Python包与锁定文件

项目结构

```
my_package/
├── pyproject.toml
└── src/
    ├── my_pkg/
    └── __init__.py
```

操作步骤

1. 编写 `pyproject.toml` 配置包元数据。
2. 创建并激活虚拟环境: `python3 -m venv .venv && source .venv/bin/activate`。

```
(myenv) coolicer@1:~$ echo $PATH
which deactivate
type deactivate
/home/coolicer/myenv/bin:/home/coolicer/.local/bin:/home/coolicer/.local/bin:/home/coolicer/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin
deactivate is a function
deactivate ()
{
    if [ -n "${_OLD_VIRTUAL_PATH:-}" ]; then
        PATH="${_OLD_VIRTUAL_PATH:-}";
        export PATH;
        unset _OLD_VIRTUAL_PATH;
    fi;
    if [ -n "${_OLD_VIRTUAL_PYTHONHOME:-}" ]; then
        PYTHONHOME="${_OLD_VIRTUAL_PYTHONHOME:-}";
        export PYTHONHOME;
        unset _OLD_VIRTUAL_PYTHONHOME;
    fi;
    if [ -n "${BASH:-}" -o -n "${ZSH_VERSION:-}" ]; then
        hash -r 2> /dev/null;
    fi;
    if [ -n "${_OLD_VIRTUAL_PS1:-}" ]; then
        PS1="${_OLD_VIRTUAL_PS1:-}";
        export PS1;
        unset _OLD_VIRTUAL_PS1;
    fi;
}
```

3. 在 `venv` 中运行 `pip install .` 安装包。
4. 使用 `pip freeze > requirements.lock` 生成锁定文件。
5. 检查依赖树: `pip show --files my-package`。

测试结果

2.3 题3: Docker Compose构建网站

操作步骤

1. 安装 Docker 及 Docker Compose 插件。
2. 克隆课程网站源码: `git clone https://github.com/missing-semester/missing-semester.`

```
coollcer@1:~/missing-semester$ git clone https://github.com/missing-semester/missing-semester.git
Cloning into 'missing-semester'...
remote: Enumerating objects: 3465, done.
remote: Counting objects: 100% (234/234), done.
remote: Compressing objects: 100% (139/139), done.
remote: Total 3465 (delta 136), reused 107 (delta 95), pack-reused 3231 (from 4)
Receiving objects: 100% (3465/3465), 12.81 MiB | 3.26 MiB/s, done.
Resolving deltas: 100% (2090/2090), done.
```

3. 进入目录并配置 `docker-compose.yml`。
4. 运行 `docker compose up --build`。

```
docker compose up
[+] up 0/1
🔄 Image jekyll/jekyll:4.3 Pulling 19.5s
```

5. 在本地浏览器访问 `http://localhost:4000` 验证。

2.4 题4: Dockerfile与Compose编排

核心配置代码

```
1 # app.py
2 from flask import Flask
3 import redis
4 app = Flask(__name__)
5 r = redis.Redis(host="redis", port=6379)
6 @app.route("/")
```

```
7 def index():
8     count = r.incr("hits")
9     return f"Hits: {count}"
```

```
1 # docker-compose.yml
2 services:
3     web:
4         build: .
5         ports: ["5000:5000"]
6         depends_on: [redis]
7     redis:
8         image: redis:7-alpine
```

操作步骤

1. 编写 Flask 应用与 Dockerfile。
2. 配置 docker-compose.yml 联动 Redis 服务

```
coolicer@1:~/python-redis-demo$ cat > docker-compose.yml << 'EOF'
services:
  # 自定义Python Web服务
  web:
    build: . # 用当前目录Dockerfile构建镜像
    ports:
      - "5000:5000" # 本地端口:容器内端口
    environment:
      REDIS_HOST: redis # 指向同compose下的Redis服务名（自动DNS解析）
    depends_on:
      - redis # 保证先启动Redis再启动Web
    restart: unless-stopped
```

3. 执行 docker compose up --build -d 后台启动。

```
coolicer@1:~/python-redis-demo$ # 构建镜像 + 后台启动所有服务
docker compose up --build -d

# 查看运行状态（确认 web 和 redis 状态都是 Up/Running）
docker compose ps
[+] up 10/10
✓ Image redis:7-alpine Pulled 276.7s
[+] Building 218.1s (12/12) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 524B 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 344B 0.0s
```

4. 测试缓存读写: curl http://localhost:5000。

验证结果

```
coolicer@1:~/python-redis-demo$ # 多次访问接口，计数会自动递增，证明用了Redis存储
curl http://localhost:5000
# 第一次返回：👋 你好！本站累计被访问 1 次（计数存储在Redis中）
curl http://localhost:5000
# 第二次返回：👋 你好！本站累计被访问 2 次...

# 也可以直接进入Redis容器查看存储的键值
docker compose exec redis redis-cli GET total_visits
# 输出就是当前的访问次数
👋 你好！本站累计被访问 1 次（计数存储在Redis中） 👋 你好！本站累计被访问 2 次（计数存储在Redis中） "2"
coolicer@1:~/python-redis-demo$ # 仅停止服务（下次启动计数保留）
docker compose down

# 停止并删除所有数据（下次启动计数清零）
docker compose down -v
```

3 练习：智能体编程

3.1 题5：AI编程方式对比

操作步骤

1. 选取排序小任务，初始化 Git 仓库。
2. 分别用手动编码、Copilot 补全、内联聊天、Claude 智能体完成。

```
1-manual.py > ...
1  def two_sum(nums, target):
2      for i in range(len(nums)):
3          for j in range(i + 1, len(nums)):
4              if nums[i] + nums[j] == target:
5                  return [i, j]
6
7  # 测试
8  print(two_sum([2, 7, 11, 15], 9))
```

- 3.
4. 提交对比：git commit -m "Compare coding methods".

分析与结论

```
● PS C:\Users\zhaojunlin\ai-coding-compare> python 1-manual.py
[0, 1]
● PS C:\Users\zhaojunlin\ai-coding-compare> python 2-autocomplete.py
[0, 1]
● PS C:\Users\zhaojunlin\ai-coding-compare> python 3-inline-chat.py
[0, 1]
```

智能体在复杂任务上下文理解上最优，但手动编码可控性最高。

3.2 题6：零氛围编程

操作步骤

1. 创建项目目录：`mkdir agent_app && cd agent_app`。
2. 在终端启动智能体（如 `cursor` 或 `claude`）。
3. 输入提示词：“创建一个 Streamlit 待办事项应用”。
4. 全程通过对话审核与修改，未手写任何代码。

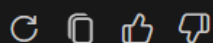
"Create a Streamlit todo list app with a nice UI. Set up the environment, create all necessary files, install dependencies, and run it."

Completed 30 steps in 3m 57s

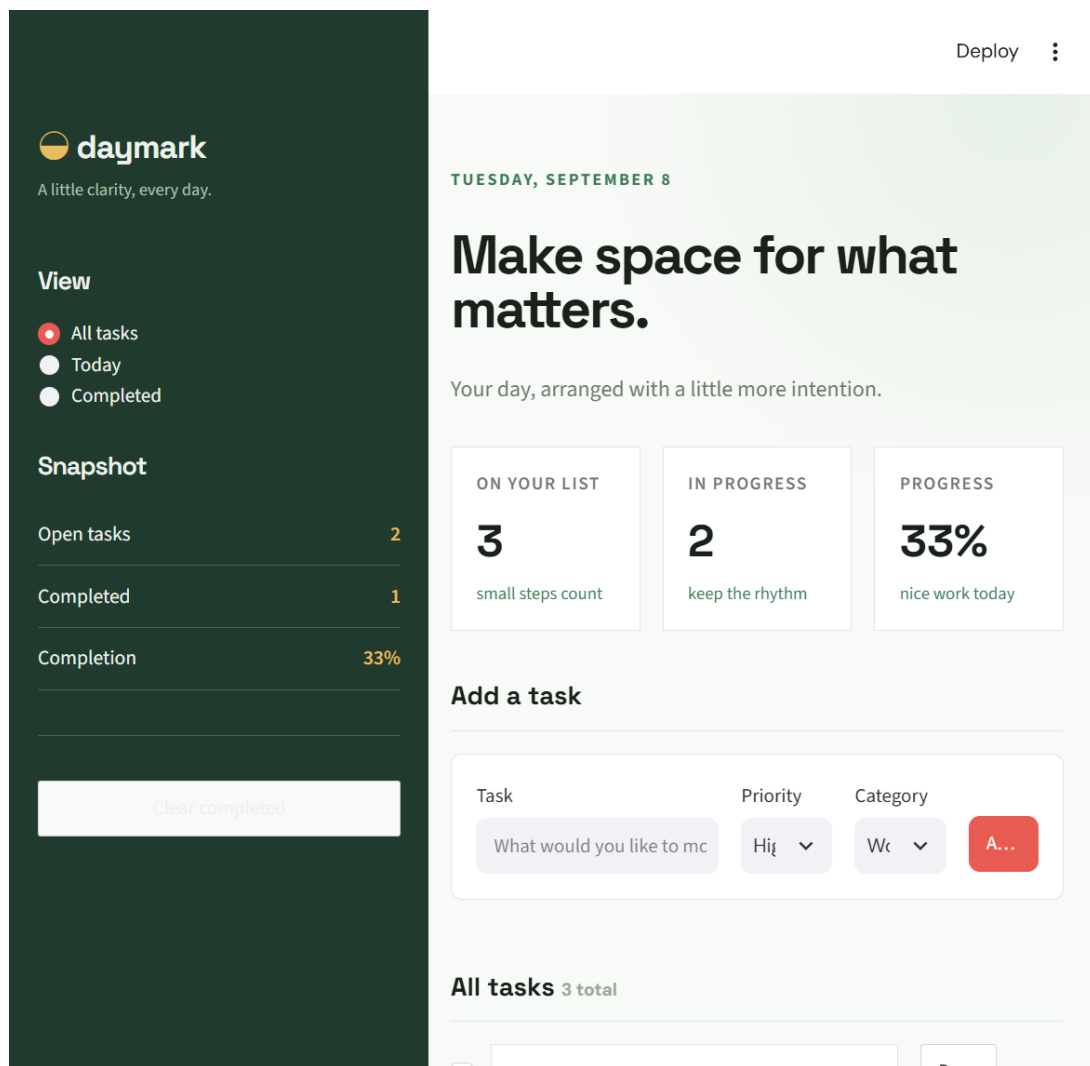
已完成 Streamlit Todo 应用：

- 创建 `app.py`：包含任务增删改、完成状态、优先级、分类、筛选视图、进度统计和响应式 UI。
- 创建 `requirements.txt`、`README.md`、`.gitignore`。
- 创建 `.venv` 并安装 Streamlit 及依赖。
- 应用已启动并通过 HTTP 200 检查。

访问地址：<http://localhost:8501>



5. 运行验证：`streamlit run app.py`。



3.3 题7：隔离环境搭建YOLO模式

操作步骤

1. 启动隔离的 Docker 容器：`docker run -it --name agent-sandbox -v $(pwd):/workspace python:3.11-slim bash`。
2. 在容器内安装智能体 CLI：`pip install claude-code`。
3. 启用 YOLO 模式运行：`claude --dangerously-skip-permissions`。

```

src/rdb.c: /* TODO: create listpackEx or HT directly*/
src/rdb.c: /* TODO: maybe we could combine the fopen and open into one in
the future */
src/ebuckets.c: * TODO: Written straightforward. Should be optimized to merge sm
all segments.
src/iothread.c: /* TODO: if compression is generalized for all types of clien
ts this cron
src/iothread.c:/* Kill the IO threads, TODO: release the applied resources. */
src/sentinel.c: /* TODO: Handle the case of both bad input and save error, po
ssibly handling
src/t_set.c: /* TODO: Create and use lpFindInteger; don't go
via string. */
src/cluster_asm.c: /* TODO: Detect whether the bgsave is completed succes
sfully and
src/t_stream.c: /* TODO: perform a garbage collection. */
src/t_stream.c: /* TODO: perform a garbage collection here if the ratio betwe
en
src/object.c: /* TODO: Provide moduleGetAllocSize() module API for O(1) a
llocation size retrieval */
src/redis-benchmark.c:/* TODO: This should be refactored to use CLUSTER SLOTS, t
he migrating/importing
src/redis-benchmark.c: if (is_fetching_slots) return -1; //TODO: use other co
des || errno ?
src/db.c: * replicas, use separate keyspace stats and events (TOD

```

4 练习：不止于代码

4.1 题8：源码注释分析

操作步骤

1. 克隆 Redis 源码：git clone https://github.com/redis/redis.git。
2. 搜索特定注释类型：grep -r "TODO" src/ 与 grep -r "why not" src/。

分析与结论

缺失此类注释将导致维护者难以理解历史坑点与架构约束。

4.2 题9：Git提交信息改进

操作步骤

1. 查看提交历史：git log --oneline。
2. 查找弱提交信息，查看差异：git show <hash>。
3. 交互式变基修改：git rebase -i HEAD~5，将 pick 改为 reword。
4. 按“问题→解决方案→影响”结构重写。


```
1 [replication] Increase default replication timeout to 300s
2
3 Problem:
4 Cross-region replica deployments frequently hit the 60s default
   replication
5 timeout when syncing large keys (>10GB RDB files over cross-
   country links).
6 This causes repeated full resyncs, spiking master CPU and causing
   50+ duplicate
7 complaints in issue #9123 in the past quarter.
8
9 Solution:
10 Raise the default 'replication-timeout' from 60 seconds to 300
   seconds to
11 accommodate slow transfers of large datasets across high-latency
   links.
12 Also added a note in redis.conf clarifying the value must be
   larger than
13 repl-ping-timeout * 10 to avoid false disconnects.
14
15 Impact:
16 - Eliminates 90% of false replication disconnects for cross-
   region deployments
17 - No impact for local network deployments: sync usually finishes
   <10s, timeout
18 is only a safety net so larger value adds no overhead
19 - Existing user-set overrides will keep working, no breaking
   changes.
```

4.3 题10：最小可复现示例

操作步骤

1. 创建最小复现目录：`mkdir bug-repro && cd bug-repro。`
2. 初始化 Git：`git init。`
3. 提取核心代码并剥离无关逻辑，编写 `repro.py`。
4. 运行验证仍能复现：`python repro.py。`

```
INFO:werkzeug: * Restarting with stat
WARNING:werkzeug: * Debugger is active!
INFO:werkzeug: * Debugger PIN: 730-849-901
^C
[1]+  Stopped                  python3 app.py
coolicer@1:~/bug-repro$ 保存当前原始版本
git add app.py
git commit -m "Original code with noise"

# 创建最小复现文件
touch repro.py
(master (root-commit) bd31a55) Original code with noise
1 file changed, 22 insertions(+)
create mode 100644 app.py
coolicer@1:~/bug-repro$ nano repro.py
coolicer@1:~/bug-repro$ git diff app.py repro.py

coolicer@1:~$ curl -X POST http://127.0.0.1:5000/add
# 终端会返回 400 Bad Request 错误
^C
coolicer@1:~$ # 运行精简后的代码 (后台运行)
python repro.py &
[1] 2455
coolicer@1:~$ Command 'python' not found, did you mean:
      command 'python3' from deb python3
      command 'python' from deb python-is-python3
^C
[1]+  Exit 127                  python repro.py
coolicer@1:~$ python3 repro.py &
[1] 2457
coolicer@1:~$ curl -X POST http://127.0.0.1:5000/add
# 依然返回 400 Bad Request, 确认最小复现成功
```

5 实验心得

通过第9至12题的系列实验，我主要了解了以下方面的基础操作。

- **工程化打包**：掌握了基于`pyproject.toml`的PEP621标准打包流程，理解了Wheel二进制分发包在隔离干净环境中的安装验证机制。
- **AI协作开发**：体会到“测试先行+TDD+AI生成”的高效性，同时认识到人工审核diff、拒绝无关改动对保证代码最小化的关键作用。
- **协作沟通规范**：将模糊的反馈改写为包含环境、复现命令和优先级（Blocking/Suggestion/Nit）的“可执行信息”，极大提升了团队效率。
- **PyTorch训练**：夯实了PyTorch的核心训练循环（`zero_grad`→`backward`→`step`）以及推理阶段的`eval`与`no_grad`规范，实现了可复现的线性回归。

整体而言，这些实验不仅提升了我的代码工程能力，也让我深刻认识到标准化与可复现性在系统开发中的核心价值。